
Robot Operating System (ROS)

Perché usare ROS? (Robot Operating System)

ROS non è un sistema operativo ma un framework modulare che risolve le difficoltà legate alla complessità, asincronia e varietà dell'hardware nei robot, facilitando lo sviluppo di software e la collaborazione.

Problemi da affrontare nei sistemi software robotici:

Lo sviluppo di software per robot è molto complesso, si parte da software di basso livello (driver) fino a percezione, ragionamento astratto, ecc.

Queste capacità vanno oltre le competenze di un singolo ricercatore.

Asincronia:

Il mondo reale è asincrono → la programmazione sequenziale tradizionale non è adatta.

Diversità dell'hardware robotico:

I robot possono avere hardware molto diversi.

Questo rende il riuso del codice non banale (perché il codice deve essere adattato a hardware differenti).

Cosa offre ROS:

Un framework che permette ai ricercatori di sviluppare rapidamente nuovi sistemi robotici.

Questo senza dover reinventare la ruota, grazie all'uso di strumenti e interfacce standard.

Perché ROS: Complessità:

Problema:

La complessità dei sistemi software per la robotica.

I robot hanno tante funzionalità → se tutto fosse scritto in un unico programma, diventerebbe ingestibile.

Soluzione proposta da ROS:

1. *Separate Processes (processi separati):*

Si scompone il software in più moduli indipendenti, ognuno con una specifica funzionalità.

2. *Un processo per ogni funzionalità:*

Es.: uno per la rilevazione ostacoli, uno per la mappa, uno per la visione ecc.

3. *Interfacce tra i processi = flussi di dati (data flows):*

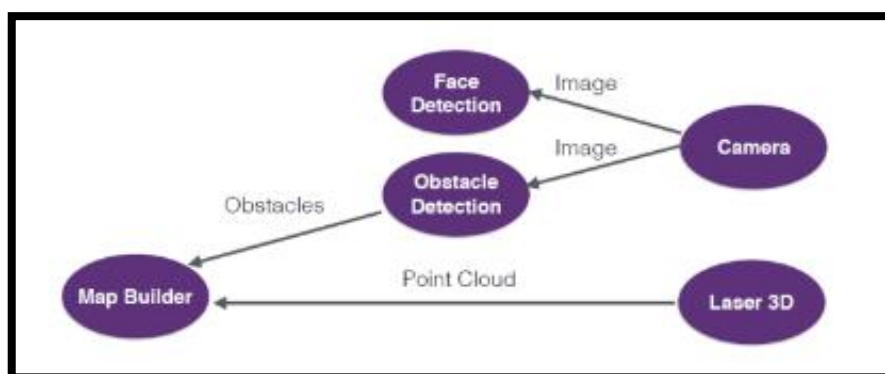
I moduli comunicano tra loro scambiandosi informazioni (immagini, nuvole di punti, ostacoli...).

4. Paradigma Publish/Subscribe:

Ogni modulo può pubblicare dati e altri moduli possono sottoscrivere a quei dati.

Ogni processo riceve solo i dati che gli servono → non deve gestire tutto.

ROS affronta la complessità creando un'**architettura modulare**, in cui ogni modulo è indipendente e comunica con gli altri tramite flussi di dati (pubblica/sottoscrive). Questo rende il sistema scalabile, riutilizzabile e più semplice da sviluppare.



Perché ROS: Asincronia

Problema:

Il mondo reale è asincrono, ma la programmazione convenzionale è sequenziale (una serie di istruzioni che si eseguono una dopo l'altra).

La sequenza rigida non si adatta alla realtà robotica, dove gli eventi avvengono in momenti imprevedibili.

Soluzione: Callbacks:

Una callback è una funzione che gestisce eventi asincroni.

Funziona come un meccanismo di feedback:

Viene chiamata automaticamente quando arrivano nuovi dati o eventi (es. un ostacolo rilevato, una nuova immagine dalla camera).

Può essere eseguita in qualsiasi momento, indipendentemente dal flusso sequenziale.

La programmazione sequenziale tradizionale → non adatta alla robotica, perché il mondo non segue un ordine prestabilito.

ROS usa le callback asincrone → il robot può reagire in tempo reale a ciò che accade (nuovi dati dai sensori, ostacoli, cambi di posizione).

Perché ROS: Diversità (Diversity):

Problema:

Esiste una grande diversità di hardware robotico:

telecamere diverse, sensori diversi, laser scanner diversi, ecc.

Ogni tipo di hardware richiederebbe codice diverso → difficile riutilizzare software tra robot differenti.

Soluzione: Astrazione dell'hardware (Hardware Abstraction)

ROS introduce uno strato di interfaccia standard tra:

1. *HW-Specific Drivers* (driver specifici per ogni dispositivo, es. camera o laser 3D).
2. *HW-Agnostic Software* (moduli software indipendenti dall'hardware, come Face Detection, Obstacle Detection, Map Builder).

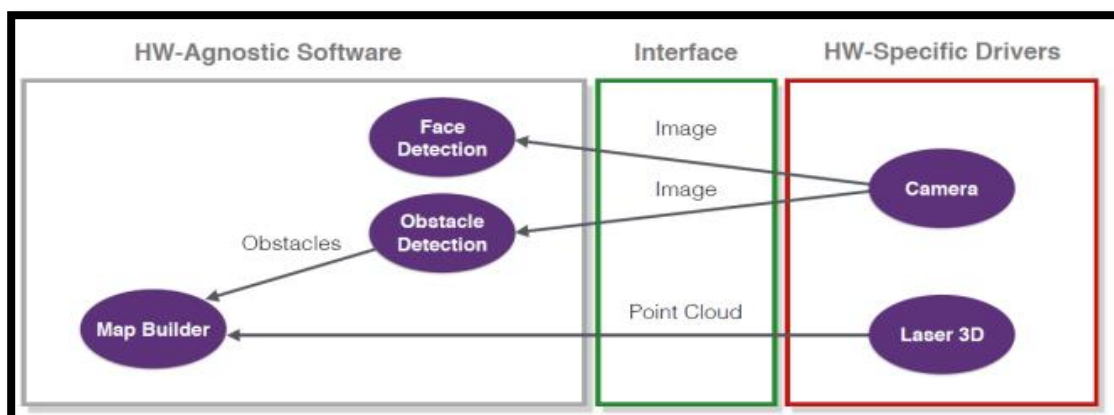
Grazie a questa separazione:

Il codice software diventa riutilizzabile.

Non importa quale camera o laser 3D venga usato: i moduli ricevono sempre immagini o nuvole di punti in un formato standard.

ROS permette di scrivere codice che funziona con diversi hardware grazie all'astrazione.

I moduli software non devono più essere riscritti per ogni nuovo dispositivo → basta cambiare il driver e tutto continua a funzionare.



Cos'è ROS?

Definizione:

ROS (Robot Operating System) non è un sistema operativo vero e proprio, ma un framework per lo sviluppo di software robotico.

Viene anche chiamato meta-operating system per robot, perché fornisce funzionalità tipiche di un sistema operativo.

Funzionalità che ROS fornisce (simili a quelle di un OS):

1. Astrazione dell'hardware (hardware abstraction) → nasconde le differenze tra dispositivi.
2. Controllo di basso livello dei dispositivi.
3. Implementazione di funzionalità comuni.
4. Comunicazione tra processi (message-passing).
5. Gestione dei pacchetti (package management).

Strumenti e librerie:

ROS offre strumenti e librerie per ottenere, costruire, scrivere ed eseguire codice su più computer contemporaneamente (distribuito).

Caratteristiche principali:

1. Architettura per la comunicazione inter-processo e inter-macchina.
2. Indipendenza dal linguaggio di programmazione → supporta C++, Python, Lisp, Java e altri.
3. Piattaforma scalabile → funziona da piccoli sistemi embedded (ARM) fino a cluster potenti (Intel Xeon).
4. Sistema standard di pacchettizzazione e compilazione.
5. Strumenti di sviluppo per il runtime e l'analisi dei dati.
6. Open-source sotto licenza BSD → il codice è libero e può essere riutilizzato/modificato senza restrizioni pesanti.

Cosa NON è ROS:

ROS è un ecosistema software open-source che fornisce astrazione, modularità e strumenti per lo sviluppo di applicazioni robotiche, funzionando come un “sistema operativo distribuito” per robot.

ROS non è nessuna delle seguenti cose:

1. Un vero sistema operativo:

ROS gira sopra un sistema operativo (es. Linux, Ubuntu), ma non lo sostituisce.

2. Un linguaggio di programmazione

ROS non è un linguaggio → usa linguaggi già esistenti come C++, Python, Lisp, Java.

3. Un ambiente di programmazione / IDE

Non è un software come Eclipse, Visual Studio o PyCharm.

Fornisce strumenti e librerie, non un editor o IDE dedicato.

4. Un’architettura real-time (tempo reale)

ROS è veloce, ma non garantisce la temporizzazione precisa delle operazioni.

Questo significa che non è adatto a sistemi che richiedono vincoli temporali stretti (es. robot chirurgici, applicazioni mission-critical).

ROS è un framework e meta-OS, ma non è un sistema operativo, un linguaggio, un IDE o una piattaforma real-time.

ROS Distributions:

Cos'è una distribuzione ROS:

Una ROS distribution è un insieme di pacchetti ROS versionato.

Ogni distribuzione contiene:

- un set di pacchetti compatibili,
- librerie,
- strumenti,
- aggiornamenti.

È molto simile a una distribuzione Linux (es. Ubuntu), che fornisce un ambiente software pronto all'uso.

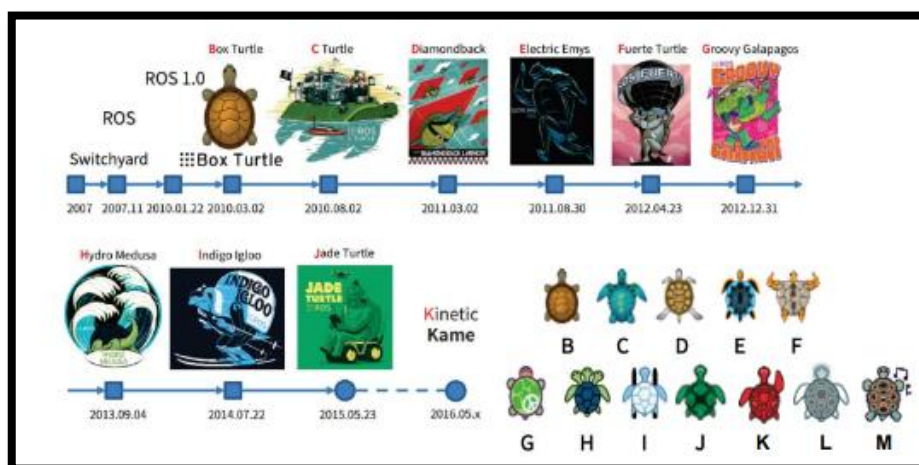
Funzione delle distribuzioni:

Garantire compatibilità e stabilità → i pacchetti inclusi sono testati per funzionare bene insieme.

Fornire aggiornamenti e nuove funzionalità in modo coordinato.

Ogni distribuzione ha un nome in codice (spesso con tema di tartarughe o animali).

Le distribuzioni ROS servono a standardizzare e semplificare l'uso di ROS, assicurando che sviluppatori e ricercatori lavorino su versioni stabili e compatibili, senza dover gestire conflitti tra pacchetti.



ROS Applications:

ROS viene utilizzato in due grandi aree applicative principali:

Perception and Mapping (Percezione e Mappatura)

1. Computer Vision (OpenCV) → riconoscimento immagini, analisi video, visione artificiale.
2. Stereo Vision / sensori di profondità (PCL) → visione 3D, calcolo distanze.
3. 2D/3D LIDAR (integrazione con Velodyne) → scansione ambientale con laser scanner.
4. Object identification → riconoscimento e classificazione degli oggetti.
5. 2D Mapping (mapserver, gmapping) → creazione di mappe bidimensionali dell'ambiente.
6. 3D Mapping (octomap) → creazione di mappe tridimensionali, utili per ambienti complessi.

Control and Planning (Controllo e Pianificazione):

1. Controllers (ROS-Industrial) → controllo di robot industriali (bracci, macchine automatizzate).
2. Motion planning (MoveIt!, OMPL) → pianificazione dei movimenti evitando ostacoli.
3. Navigation stack (2d_nav) → navigazione autonoma di robot mobili in ambienti 2D.
4. Manipulation and grasping → manipolazione e presa di oggetti con bracci robotici.

ROS fornisce strumenti pronti all'uso per permettere ai robot di:

- percepire l'ambiente (visione, LIDAR, mappatura),
- prendere decisioni e muoversi (controllo, navigazione, pianificazione dei movimenti),
- interagire con gli oggetti (manipolazione, grasping).

ROS Core:

Il ROS core è l'insieme di nodi e programmi che costituiscono la base necessaria per il funzionamento di un sistema ROS.

ROS Master:

Ruolo principale: gestisce la comunicazione tra i nodi.

Funzioni:

- Negozia le connessioni di comunicazione tra i vari processi.
- La comunicazione in ROS è peer-to-peer (i nodi parlano direttamente tra loro), ma serve un lookup service (tipo rubrica) per farli incontrare → questo ruolo è svolto dal Master.
- Registra e cerca i nomi delle risorse (nodi, topic, servizi) → funziona come un name server.

In pratica: senza rosmaster i nodi non saprebbero come trovarsi e comunicare.

ROS Parameter Server:

Un dizionario condiviso di parametri di configurazione e dati variabili.

Consente di memorizzare e recuperare valori in runtime attraverso chiavi.

Esempio: parametri di calibrazione di una camera o costanti di controllo.

rosout (logging node)

Sistema centralizzato di **logging** (stdout/stderr).

Serve per avere un meccanismo unico e leggibile di messaggi, debug ed errori.

Comando da avviare per iniziare a usare ROS: `$ roscore`

Avvia insieme ROS Master, Parameter Server, rosout

È indispensabile: senza roscore, i nodi non possono comunicare.

Il ROS Core è il nucleo che rende possibile la comunicazione e la gestione delle risorse in ROS.

Nodes:

Definizione:

Un node è un file eseguibile all'interno di un pacchetto ROS → rappresenta un processo che svolge calcoli o funzioni specifiche.

I nodi usano una libreria client ROS per comunicare tra loro.

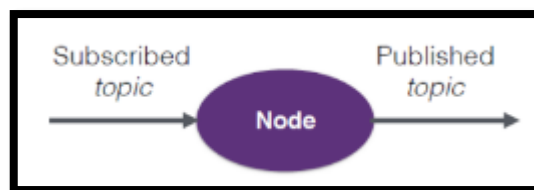
Comunicazione:

I nodi comunicano attraverso i Topic:

Un nodo può pubblicare dati su un topic (es. una camera pubblica immagini).

Un nodo può sottoscrivere (subscribe) a un topic per ricevere dati (es. un algoritmo di face detection riceve le immagini dalla camera).

Oltre ai topic, i nodi possono anche usare i Service, cioè chiamate sincrone a funzioni di altri nodi.



Comandi utili:

rostopic serve per avere informazioni sui nodi attivi.

Esempi:

- `rostopic list` → elenca tutti i nodi in esecuzione.
- `rostopic info <node_name>` → mostra i dettagli di un nodo.
- `rostopic ping <node_name>` → controlla se un nodo risponde.

roslaunch PACKAGE EXECUTABLE: Serve per avviare un nodo.

Cerca dentro il **PACKAGE** l'eseguibile specificato e lo esegue.

Esempio: `roslaunch turtlesim turtlesim_node`

Nodes - Tutorial (Turtlesim)

Cos'è Turtlesim?

Un simulatore grafico semplice fornito da ROS.

Usato come strumento didattico per imparare come funzionano nodi, topic e servizi in ROS.

Comandi principali:

1. Avviare un nodo: `$ rosrun turtlesim turtlesim_node`

Avvia la finestra grafica con una tartaruga che puoi controllare.

Questo crea un nodo chiamato turtlesim_node.

2. Elenco dei nodi attivi: `$ rosnode list`

Mostra tutti i nodi ROS attualmente in esecuzione.

Vedrai turtlesim_node e altri come /rosout.

3. informazioni su un nodo: `$ rosnode info /rosout`

Mostra dettagli su un nodo, come:

su quali topic pubblica,

a quali topic è iscritto,

chi sono i suoi peer.

4. Rinominare un nodo (remapping): `$ rosrun turtlesim turtlesim_node _name:=my_turtle`

Avvia turtlesim_node ma rinomina il nodo in my_turtle.

Utile per distinguere nodi multipli che eseguono lo stesso eseguibile.

5. Ping di un nodo: `$ rosnode ping my_turtle`

Verifica se un nodo è attivo e risponde.

Turtlesim è un **esempio pratico** per imparare a:

1. avviare nodi (roslaunch),
2. elencarli (roslaunch list),
3. ispezionarli (roslaunch info),
4. rinominarli (__name),
5. testarli (roslaunch ping).



Topics and Messages:

Topics:

I topics permettono a più nodi di comunicare tra loro (many-to-many communication streams).

L'idea è quella di separare (decoupling) chi produce i dati da chi li consuma.

ROS supporta trasporto via TCP/IP e UDP.

Ogni topic è fortemente tipizzato → significa che accetta solo messaggi di un certo tipo (ROS message).

Messages:

Un message è una struttura dati tipizzata, composta da campi (es. interi, float, stringhe, array).

I messaggi possono contenere sia tipi primitivi sia strutture annidate.

Comunicazione:

Node1 → pubblica un messaggio su un topic.

Node2 → si iscrive allo stesso topic e riceve il messaggio.

Questo rende i nodi indipendenti: non devono conoscersi direttamente, basta condividere il topic.

I topics sono i canali di comunicazione tra nodi.

I messages sono i dati strutturati che viaggiano sui topic.

Comandi utili

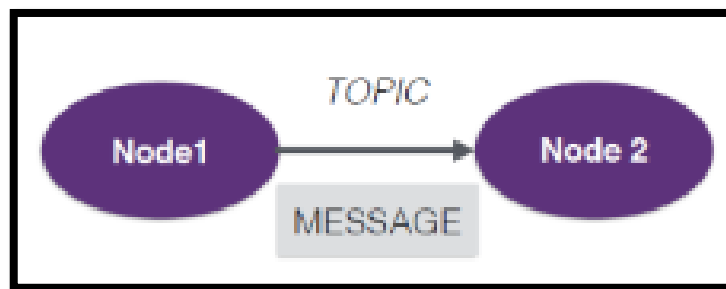
Rostopic:

- rostopic list → mostra tutti i topic attivi.
- rostopic echo <topic> → mostra i messaggi che passano su un topic.
- rostopic type <topic> → mostra il tipo di messaggio del topic.
- rostopic pub <topic> <msg_type> <data> → permette di pubblicare manualmente un messaggio su un topic.

Rosmsg:

- rosmsg show <msg> → mostra la struttura di un tipo di messaggio.

Con rostopic e rosmsg si esplorano e si monitorano gli scambi di messaggi.



Topics and Messages – Tutorial:

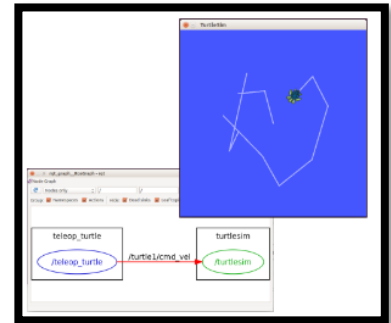
Avvio dei nodi:

1. Avvio della tartaruga (simulatore): `$ rosrun turtlesim turtlesim_node`
2. Controllo manuale con la tastiera: `$ rosrun turtlesim turtle_teleop_key`
3. Visualizzazione grafica della rete di nodi e topic: `$ rosrun rqt_graph rqt_graph`

Questo genera un grafico dinamico che mostra chi pubblica e chi si iscrive ai topic.

Nell'immagine:

- `teleop_turtle` → Publisher (invia comandi di velocità).
- `/turtle1/cmd_vel` → Topic (canale di comunicazione).
- `turtlesim` → Subscriber (riceve i comandi e muove la tartaruga).



Interazione con i topic:

1. Visualizzare i messaggi che passano su un topic: `$ rostopic echo /turtle1/cmd_vel`

Mostra i messaggi di velocità inviati (del tipo `geometry_msgs/Twist`).

2. Vedere il tipo di messaggio: `$ rostopic type /turtle1/cmd_vel`

Output: `geometry_msgs/Twist`

3. Vedere la struttura del messaggio: `$ rosmmsg show geometry_msgs/Twist`

Mostra i campi `linear` e `angular`.

4. Pubblicare un messaggio singolo (single command):

```
$ rostopic pub -1 /turtle1/cmd_vel geometry_msgs/Twist -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'
```

La tartaruga si muove in avanti con velocità lineare 2.0 e ruota con velocità angolare 1.8.

5. Pubblicare comandi continui:

```
$ rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 0.5]'
```

Invia comandi ogni secondo (`-r 1`) → la tartaruga continua a muoversi e girare.

Master Name Service Example:

Passaggi principali:

1. Pubblicazione annunciata (advertise)

La Camera dice al Master: “Voglio pubblicare immagini sul topic /images”.

Il Master registra questa informazione, ma al momento nessuno è interessato → quindi nessun dato viene ancora inviato.

2. Sottoscrizione (subscribe)

L'Image Viewer comunica al Master: “Voglio sottoscrivermi al topic /images”.

Ora il Master sa che:

- Camera è publisher del topic /images,
- Image Viewer è subscriber dello stesso topic.

3. Comunicazione peer-to-peer:

Una volta che il Master ha fatto incontrare publisher e subscriber, i due nodi stabiliscono una connessione diretta (peer-to-peer).

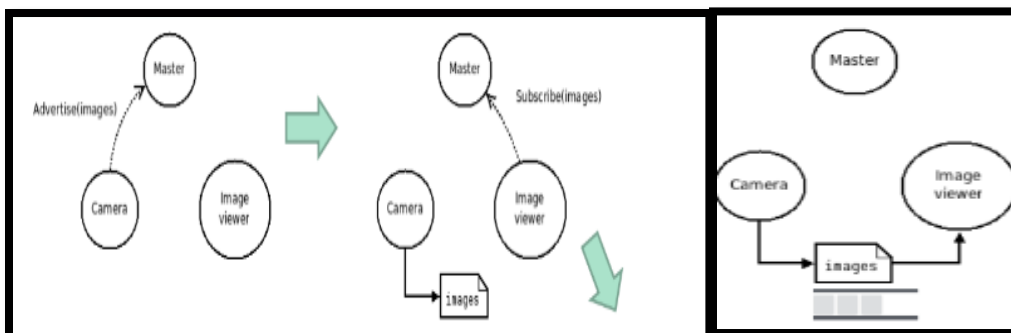
Da quel momento:

- la Camera invia immagini,
- l'Image Viewer le riceve.

Il Master non trasporta i dati → serve solo per far sì che i nodi si “trovino”.

Il ROS Master è come una rubrica telefonica:

Non trasporta i dati ma registra chi pubblica e chi si iscrive mettendo in contatto i nodi per permettere loro di scambiarsi messaggi direttamente.



Services:

Definizione:

I services sono un altro meccanismo di comunicazione tra nodi, diverso dai topics.

Si basano sul modello request/reply (richiesta/risposta).

Mentre i topics sono asincroni e unidirezionali (publish/subscribe), i services sono sincroni e bidirezionali:

- Un nodo fa una richiesta.
- Un altro nodo risponde con una risposta.

Struttura:

Ogni service è definito da **due messaggi**:

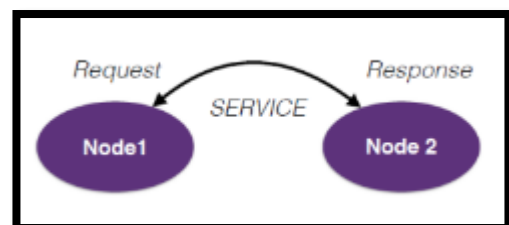
- uno per la request (richiesta),
- uno per la response (risposta).

Un nodo può offrire un servizio (server), registrandolo sotto un nome.

Un altro nodo può chiamare quel servizio (client), inviando la richiesta e aspettando la risposta.

Esempio nello schema:

- Node1 → invia una richiesta.
- Node2 → elabora e risponde.



La comunicazione avviene **peer-to-peer**, come nei topics, ma in maniera sincrona.

Comandi utili:

Roservice:

- `roscall` list → mostra tutti i servizi disponibili.
- `roscall` call <srv> → chiama un servizio.
- `roscall` type <srv> → mostra il tipo di messaggio usato dal servizio.

Rosrv:

- `roscall` show <srv> → mostra la struttura del messaggio usato da un service (request/response).

I topic → per scambio continuo di dati (stream asincrono).

I service → per richieste specifiche con risposta (sincrono, tipo funzione remota).

Services – Tutorial:

1. Elenco dei servizi disponibili: `$ roservice list`

Mostra tutti i servizi attivi nel sistema.

2. Verificare il tipo di un servizio: `$ roservice type /clear`

Questo non richiede argomenti (no args).

`$ roservice type /spawn`

Questo invece richiede argomenti (with args).

3. Per vedere la struttura del messaggio: `$ rossrv show turtlesim/Spawn`

Mostra i campi richiesti dal servizio /spawn (coordinate X, Y, angolo, nome).

4. Cancellare lo schermo di Turtlesim: `$ roservice call /clear`

5. Creare una nuova tartaruga: `$ roservice call /spawn 2 2 0.2 ""`

Qui stiamo creando una nuova tartaruga in posizione (x=2, y=2) con angolo 0.2 rad.

Differenza tra paradigmi ROS:

Publish/Subscribe → comunicazione continua, asincrona, senza bisogno che i nodi si conoscano.

Service → comunicazione sincrona, richiesta/risposta, utile per comandi specifici.

Con i Services puoi chiedere azioni specifiche al robot (es. spawn, clear, reset), mentre con i Topics controlli flussi continui di dati (es. velocità, sensori).

Parameters:

Parameter Server:

È un server centrale (parte del Master) che permette di memorizzare e gestire dati usando delle chiavi.

Serve per condividere informazioni tra i nodi, ad esempio:

- costanti di configurazione,
- parametri di calibrazione (es. camera),
- impostazioni di controllo.

Caratteristiche dei parametri:

1. Nomi → seguono le convenzioni di naming di ROS.
2. Gerarchia (hierarchy) → i parametri possono essere organizzati ad albero evitando i conflitti di nomi (tipo filesystem).
Permette di accedere ai parametri singolarmente o come parte di un gruppo.

3. Tipi di dati supportati:

interi a 32 bit, booleani, stringhe, numeri in virgola mobile (double), liste, strutture.

Comando principale:

rosparam → per interagire con il Parameter Server:

`rosparam set <key> <value>` → imposta un parametro.

`rosparam get <key>` → legge un parametro.

`rosparam list` → elenca tutti i parametri attivi.

Il Parameter Server è una sorta di database globale condiviso dai nodi ROS, utile per configurazioni e parametri modificabili anche a runtime.

Parameters – Tutorial:

1. Vedere i parametri attivi: `$ rosparam list`

Mostra tutti i parametri attualmente disponibili sul Parameter Server.

2. Modificare un parametro: `$ rosparam set background_r 150`

Imposta il valore del canale rosso dello sfondo a 150.

N.B) La modifica non è immediatamente visibile → bisogna usare un servizio per aggiornare la finestra.

3. Applicare le modifiche: `$ rosservice call clear`

Cancella lo schermo e ridisegna lo sfondo con i nuovi valori dei parametri → qui vediamo il colore cambiare in viola.

4. Leggere un parametro: `$ rosparam get background_g`

Restituisce il valore attuale del canale verde dello sfondo.

`$ rosparam get /`

Restituisce tutti i parametri presenti sul Parameter Server, in formato ad albero (gerarchia).

Esempio pratico:

Impostando:

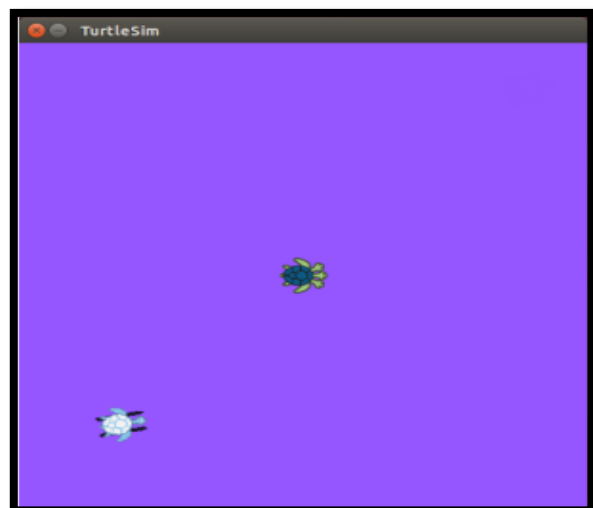
```
rosparam set background_r 150
```

```
rosparam set background_g 50
```

```
rosparam set background_b 200
```

```
rosservice call clear
```

Lo sfondo diventerà viola, come nell'immagine della slide.



Con *rosparam* possiamo leggere e modificare configurazioni a runtime, mentre con *rosservice call clear* applichiamo i cambiamenti in Turtlesim.

Launch files:

Cosa sono:

I launch file servono per avviare più nodi contemporaneamente, sia in locale che in remoto.

Permettono anche di impostare parametri nel Parameter Server al momento del lancio.

Sono scritti in XML con estensione .launch.

Comando principale: `$ roslaunch [package] [filename.launch]`

Avvia i nodi e i parametri definiti nel file di configurazione .launch.

Relazione con roscore:

roscore è in realtà un caso speciale di roslaunch:

Avvia automaticamente il ROS Master, il Parameter Server e il logging (rosout).

Funzionalità principali:

1. Namespaces (ns) → organizzano i nodi in spazi di nomi (utile per evitare conflitti).
2. Node → specifica quale nodo avviare (pacchetto, nome, tipo).
3. Remap → permette di rinominare topic o servizi.
4. Include → consente di richiamare altri file .launch.

I launch file sono fondamentali per gestire applicazioni ROS complesse, perché:

1. avviano più nodi in un solo comando,
2. impostano parametri,
3. gestiscono namespaces e remapping,
4. possono includere altri file di configurazione.

```
<launch>

<group ns="namespace_1">
  <node pkg="package_name_1" name="node_1_instance_1" type="node_1"/>
</group>

<group ns="namespace_2">
  <node pkg="package_name_1" name="node_1_instance_2" type="node_1"/>
</group>

<node pkg="package_name_2" name="node_2" type="node_2">
  <remap from="input" to="node_2/input"/>
  <remap from="output" to="node_2/output"/>
</node>

<include file="$(find package_name_3)/launch/filelaunch.launch" />

</launch>
```

From turtlesim to... real robot:

L'idea:

ROS permette di sviluppare e testare algoritmi prima in simulazione (con Turtlesim, Gazebo, ecc.) e poi sul robot reale, senza dover riscrivere il codice.

Questo è possibile grazie all'astrazione di ROS (topics, services, parameters).

Esempio: Turtlesim:

1. Avvio di ROS core (roscore).
2. Avvio del simulatore Turtlesim (turtlesim_node).
3. Controllo della tartaruga tramite joystick o tastiera (turtle_teleop_key).

Caso reale: Robot fisico:

1. Avvio del robot reale.
2. Esecuzione dei nodi del robot (driver per sensori, attuatori, telecamere...).
3. Controllo del robot tramite joystick (o altri input).

Confronto:

Turtlesim = simulazione semplice → muovi la tartaruga su schermo.

Robot reale = stesso paradigma → muovi un vero robot con gli stessi comandi.

ROS permette un passaggio naturale dal simulatore al robot reale:

- stessi concetti (nodi, topic, servizi, parametri),
- stesso codice di controllo,
- cambia solo l'“interfaccia hardware” (driver vs simulatore).